■ Python Intermediate → Advanced

Practice Roadmap

Writing by Eng Muhammad Nouman

1. OOP & Python Data Model

- __init__, instance/class attributes
- Inheritance & composition
- __str__, __repr__
- __len__, __getitem__, __contains__
- Operator overloading
- __eq__, __lt__, __hash__
- Properties
- Abstract classes
- Dataclasses
- Custom objects behaving like built-in types

2. Iterators & Generators

- Iterable vs iterator
- iter() / next()
- Custom iterator
- yield
- Generator functions
- Generator expressions
- send()
- Practical lazy processing

3. Comprehensions

- List/set/dict comprehensions
- Nested comprehensions
- Conditions
- Comprehensions + functions
- When not to use comprehensions

4. Decorators & Closures

- Functions as objects
- Nested functions
- Closure
- nonlocal
- Basic decorator

- *args, **kwargs
- functools.wraps
- Parametrized decorators
- Retry decorator
- Logging decorator
- Timing decorator
- Authentication/permission-style decorators

5. Exception Handling

- try/except
- Multiple exceptions
- else
- finally
- raise
- Custom exceptions
- Exception chaining
- Designing useful error handling

6. Context Managers

- with
- __enter__
- __exit__
- Custom context manager
- contextlib
- @contextmanager
- Practical transaction/file/resource examples

7. Async Fundamentals

- Synchronous vs asynchronous
- async
- await
- Coroutine
- Event loop
- asyncio
- create_task
- gather
- Async HTTP/API concepts
- When async actually helps

8. Memory & Performance

- Mutable vs immutable

- References
- Shallow/deep copy
- Garbage collection
- Generators vs lists
- Memory-efficient code
- timeit
- Basic profiling
- Big-O practical understanding